DATA STRUCTURES AND ALGORITHMS LAB

Practical Problem Statements

Part A: C++ Programs (Questions 1–8)

Part B: Python Programs (Questions 9–20)

Each question includes an algorithm first, followed by a simple program.

Student Name: _____

Roll Number: _____

Class / Division: _____

PART A — C++

Questions 1–8

Question 1. Student database using dynamic memory and Linear Search

Algorithm

1. Start.
2. Read the number of students n.
3. Allocate memory dynamically for n student records.
4. Read each student's ID, name and CGPA.
5. Read the ID to search.
6. Compare the ID with each record using Linear Search.
7. If a match is found, display that student's details and stop searching.
8. If no match is found, display “Student not found”.
9. Release dynamically allocated memory and stop.

Code

```
#include <iostream>
using namespace std;

struct Student {
    int id;
    string name;
    float cgpa;
};

int main() {
    int n, key;
    cout << "Enter number of students: ";
    cin >> n;

    Student *s = new Student[n];

    for (int i = 0; i < n; i++) {
        cout << "Enter ID, name and CGPA: ";
        cin >> s[i].id >> s[i].name >> s[i].cgpa;
    }

    cout << "Enter ID to search: ";
    cin >> key;

    for (int i = 0; i < n; i++) {
        if (s[i].id == key) {
            cout << s[i].id << " " << s[i].name
                 << " " << s[i].cgpa;
            delete[] s;
            return 0;
        }
    }

    cout << "Student not found";
    delete[] s;
    return 0;
}
```

Question 2. Binary Search on sorted student IDs

Algorithm

1. Start.
2. Store student IDs in an array and sort them in ascending order.
3. Read the ID to search.
4. Set low to the first index and high to the last index.
5. Find mid = (low + high) / 2.
6. If the middle ID equals the key, report that it is found.
7. If the key is greater, search the right half; otherwise search the left half.
8. Repeat while low <= high.
9. If the search ends without a match, report that the ID is not found.

Code

```
#include <iostream>
#include <algorithm>
using namespace std;

int main() {
    int a[5] = {105, 102, 104, 101, 103};
    sort(a, a + 5);

    int key, low = 0, high = 4;
    cout << "Enter ID: ";
    cin >> key;

    while (low <= high) {
        int mid = (low + high) / 2;

        if (a[mid] == key) {
            cout << "ID found";
            return 0;
        }
        else if (a[mid] < key)
            low = mid + 1;
        else
            high = mid - 1;
    }

    cout << "ID not found";
    return 0;
}
```

Question 3. Sort student records by CGPA using Bubble Sort

Algorithm

1. Start.
2. Read the ID, name and CGPA of each student.
3. Compare adjacent student records by CGPA.
4. If the first CGPA is greater than the next CGPA, swap the complete records.
5. Repeat the passes until the records are sorted in ascending order of CGPA.
6. Display the sorted student records.
7. Stop.

Code

```
#include <iostream>
using namespace std;

struct Student {
    int id;
    string name;
    float cgpa;
};

int main() {
    Student s[3], temp;

    for (int i = 0; i < 3; i++) {
        cout << "Enter ID, name and CGPA: ";
        cin >> s[i].id >> s[i].name >> s[i].cgpa;
    }

    for (int i = 0; i < 2; i++) {
        for (int j = 0; j < 2 - i; j++) {
            if (s[j].cgpa > s[j + 1].cgpa) {
                temp = s[j];
                s[j] = s[j + 1];
                s[j + 1] = temp;
            }
        }
    }

    for (int i = 0; i < 3; i++)
        cout << s[i].id << " " << s[i].name
             << " " << s[i].cgpa << endl;

    return 0;
}
```

Question 4. Binary Search on sorted student IDs (same as Question 2)

Algorithm

Use the same algorithm as Question 2: sort the IDs in ascending order, repeatedly compare the key with the middle element, and narrow the search to the left or right half until the ID is found or the range becomes empty.

Code

The code for Question 2 also satisfies Question 4.

Question 5. Stack ADT using a singly linked list

Algorithm

1. Start with top = NULL.
2. For PUSH, create a node, store the value, link it to the current top and update top.
3. For POP, check whether top is NULL. If it is, display stack underflow.
4. Otherwise, remove the top node, update top and free the removed node.
5. For DISPLAY, traverse from top to NULL and print each value.
6. Stop.

Code

```
#include <iostream>
using namespace std;

struct Node {
    int data;
    Node *next;
};

Node *top = NULL;

void push(int x) {
    Node *n = new Node;
    n->data = x;
    n->next = top;
    top = n;
}

void pop() {
    if (top == NULL) {
        cout << "Stack underflow\n";
        return;
    }

    Node *temp = top;
    cout << "Deleted: " << top->data << endl;
    top = top->next;
    delete temp;
}

void display() {
    Node *p = top;
    while (p != NULL) {
        cout << p->data << " ";
        p = p->next;
    }
    cout << endl;
}

int main() {
    push(10);
    push(20);
    push(30);
    display();
    pop();
    display();
    return 0;
}
```

Question 6. Convert infix expression to postfix using a stack

Algorithm

1. Start and read a valid infix expression.
2. Scan the expression from left to right.
3. Append operands directly to the postfix expression.
4. Push '(' onto the stack.
5. For ')', pop operators to the output until '(' is found; discard the parentheses.
6. For an operator, pop higher- or equal-precedence operators from the stack, then push the current operator.
7. Pop all remaining operators to the output.
8. Display the postfix expression and stop.

Code

```
#include <iostream>
#include <stack>
#include <cctype>
using namespace std;

int prec(char c) {
    if (c == '^') return 3;
    if (c == '*' || c == '/') return 2;
    if (c == '+' || c == '-') return 1;
    return 0;
}

int main() {
    string s, p = "";
    stack<char> st;

    cout << "Enter infix: ";
    cin >> s;

    for (char c : s) {
        if (isalnum(c))
            p += c;
        else if (c == '(')
            st.push(c);
        else if (c == ')') {
            while (!st.empty() && st.top() != '(') {
                p += st.top();
                st.pop();
            }
            if (!st.empty()) st.pop();
        }
        else {
            while (!st.empty() && st.top() != '(' &&
                   prec(st.top()) >= prec(c)) {
                p += st.top();
                st.pop();
            }
            st.push(c);
        }
    }

    while (!st.empty()) {
        p += st.top();
        st.pop();
    }

    cout << "Postfix: " << p;
    return 0;
}
```

Question 7. Convert infix expression to prefix using a stack

Algorithm

1. Start and read a valid infix expression.
2. Reverse the expression and interchange '(' with ')'.
3. Scan the modified expression from left to right.
4. Append operands to the output and push opening parentheses.
5. For a closing parenthesis, pop operators until an opening parenthesis is found.
6. For an operator, pop operators of higher precedence, then push the current operator.
7. Pop remaining operators and reverse the output.
8. Display the prefix expression and stop.

Code

```
#include <iostream>
#include <stack>
#include <algorithm>
#include <cctype>
using namespace std;

int prec(char c) {
    if (c == '^') return 3;
    if (c == '*' || c == '/') return 2;
    if (c == '+' || c == '-') return 1;
    return 0;
}

int main() {
    string s, p = "";
    stack<char> st;

    cout << "Enter infix: ";
    cin >> s;
    reverse(s.begin(), s.end());

    for (char &c : s) {
        if (c == '(') c = ')';
        else if (c == ')') c = '(';
    }

    for (char c : s) {
        if (isalnum(c))
            p += c;
        else if (c == '(')
            st.push(c);
        else if (c == ')') {
            while (!st.empty() && st.top() != '(') {
                p += st.top();
                st.pop();
            }
            if (!st.empty()) st.pop();
        }
        else {
            while (!st.empty() && st.top() != '(' &&
                   prec(st.top()) > prec(c)) {
                p += st.top();
                st.pop();
            }
            st.push(c);
        }
    }

    while (!st.empty()) {
        p += st.top();
        st.pop();
    }

    reverse(p.begin(), p.end());
    cout << "Prefix: " << p;
    return 0;
}
```

Question 8. Circular Queue using an array of size 5

Algorithm

1. Initialize front = rear = -1.
2. ENQUEUE: if (rear + 1) % 5 == front, report queue full. Otherwise, set front to 0 if the queue is empty, move rear circularly and insert the value.
3. DEQUEUE: if front == -1, report queue empty. Otherwise, remove the front value.
4. If front == rear, reset both to -1; otherwise move front circularly.
5. DISPLAY values from front to rear, wrapping around the array when needed.
6. Stop.

Code

```
#include <iostream>
using namespace std;

int q[5], front = -1, rear = -1;

void enqueue(int x) {
    if ((rear + 1) % 5 == front) {
        cout << "Queue is full\n";
        return;
    }
    if (front == -1) front = 0;
    rear = (rear + 1) % 5;
    q[rear] = x;
}

void dequeue() {
    if (front == -1) {
        cout << "Queue is empty\n";
        return;
    }
    cout << "Deleted: " << q[front] << endl;
    if (front == rear)
        front = rear = -1;
    else
        front = (front + 1) % 5;
}

void display() {
    if (front == -1) {
        cout << "Queue is empty\n";
        return;
    }
    int i = front;
    while (true) {
        cout << q[i] << " ";
        if (i == rear) break;
        i = (i + 1) % 5;
    }
    cout << endl;
}

int main() {
    enqueue(10);
    enqueue(20);
    enqueue(30);
    display();
    dequeue();
    display();
    return 0;
}
```

PART B — PYTHON

Questions 9–20

Question 9. BST insertion and inorder, preorder and postorder traversals

Algorithm

1. Start with an empty BST.
2. Insert each integer: smaller values go to the left subtree and larger values go to the right subtree. Ignore duplicates.
3. INORDER: traverse left subtree, node, right subtree.
4. PREORDER: visit node, left subtree, right subtree.
5. POSTORDER: traverse left subtree, right subtree, node.
6. Display all three traversals and stop.

Code

```
class Node:
    def __init__(self, data):
        self.data = data
        self.left = None
        self.right = None

def insert(root, x):
    if root is None:
        return Node(x)
    if x < root.data:
        root.left = insert(root.left, x)
    elif x > root.data:
        root.right = insert(root.right, x)
    return root

def inorder(root):
    if root:
        inorder(root.left)
        print(root.data, end=" ")
        inorder(root.right)

def preorder(root):
    if root:
        print(root.data, end=" ")
        preorder(root.left)
        preorder(root.right)

def postorder(root):
    if root:
        postorder(root.left)
        postorder(root.right)
        print(root.data, end=" ")

root = None
for x in [50, 30, 70, 20, 40]:
    root = insert(root, x)

print("Inorder:")
inorder(root)
print("\nPreorder:")
preorder(root)
print("\nPostorder:")
postorder(root)
```

Question 10. Insert and search in a BST

Algorithm

1. Start with an empty BST.
2. Insert values according to BST rules: smaller values go left and larger values go right.
3. Read the key to search.
4. If the current node is NULL, the key is absent.
5. If the current node contains the key, report that it is present.
6. Otherwise, continue in the left or right subtree based on the key.
7. Display the result and stop.

Code

```
class Node:
    def __init__(self, data):
        self.data = data
        self.left = None
        self.right = None

def insert(root, x):
    if root is None:
        return Node(x)
    if x < root.data:
        root.left = insert(root.left, x)
    elif x > root.data:
        root.right = insert(root.right, x)
    return root

def search(root, key):
    if root is None:
        return False
    if root.data == key:
        return True
    if key < root.data:
        return search(root.left, key)
    return search(root.right, key)

root = None
for x in [50, 30, 70, 20, 40]:
    root = insert(root, x)

key = int(input("Enter key to search: "))
if search(root, key):
    print("Key found")
else:
    print("Key not found")
```

Question 11. Delete a specified key from a BST

Algorithm

1. Start and insert values into a BST.
2. Search for the key to delete.
3. If the key is absent, leave the tree unchanged.
4. If the node is a leaf, remove it.
5. If it has one child, replace it with its child.
6. If it has two children, find the smallest value in its right subtree, copy that value to the node and delete the duplicate successor node.
7. Display the resulting BST using inorder traversal.

Code

```
class Node:
    def __init__(self, data):
        self.data = data
        self.left = None
        self.right = None

def insert(root, x):
    if root is None:
        return Node(x)
    if x < root.data:
        root.left = insert(root.left, x)
    elif x > root.data:
        root.right = insert(root.right, x)
    return root

def delete(root, key):
    if root is None:
        return None
    if key < root.data:
        root.left = delete(root.left, key)
    elif key > root.data:
        root.right = delete(root.right, key)
    else:
        if root.left is None:
            return root.right
        if root.right is None:
            return root.left
        temp = root.right
        while temp.left:
            temp = temp.left
        root.data = temp.data
        root.right = delete(root.right, temp.data)
    return root

def inorder(root):
    if root:
        inorder(root.left)
        print(root.data, end=" ")
        inorder(root.right)

root = None
for x in [50, 30, 70, 20, 40]:
    root = insert(root, x)

key = int(input("Enter value to delete: "))
root = delete(root, key)
print("BST after deletion:")
inorder(root)
```

Question 12. Find BST depth and display all leaf nodes

Algorithm

1. Start and create a BST.
2. To find depth, return -1 for an empty tree.
3. For a non-empty node, calculate 1 + maximum of the left and right subtree depths.
4. A leaf node has no left or right child; display its value.
5. Display the depth and all leaf nodes, then stop.

Code

```
class Node:
    def __init__(self, data):
        self.data = data
        self.left = None
        self.right = None

def insert(root, x):
    if root is None:
        return Node(x)
    if x < root.data:
        root.left = insert(root.left, x)
    elif x > root.data:
        root.right = insert(root.right, x)
    return root

def depth(root):
    if root is None:
        return -1
    return 1 + max(depth(root.left), depth(root.right))

def leaves(root):
    if root:
        if root.left is None and root.right is None:
            print(root.data, end=" ")
        leaves(root.left)
        leaves(root.right)

root = None
for x in [50, 30, 70, 20, 40]:
    root = insert(root, x)

print("Depth:", depth(root))
print("Leaf nodes:")
leaves(root)
```

Question 13. Kruskal's Minimum Spanning Tree using weighted edges

Algorithm

1. Represent campus locations as vertices and paths as weighted edges.
2. Sort all edges in increasing order of weight.
3. Initialize each vertex as a separate set.
4. Consider each edge in sorted order.
5. If its endpoints belong to different sets, select the edge and merge the sets.
6. Stop after selecting V - 1 edges or when all edges have been checked.
7. Display the selected edges and total weight. If the graph is disconnected, a spanning tree does not exist.

Code

```
# Each edge is (source, destination, weight)
edges = [
    (2, 3, 1),
    (0, 1, 2),
    (1, 2, 3),
    (0, 2, 4),
    (2, 4, 5),
    (3, 4, 6)
]

parent = list(range(5))

def find(x):
    if parent[x] != x:
        parent[x] = find(parent[x])
    return parent[x]

edges.sort(key=lambda e: e[2])
total = 0
count = 0

print("Selected edges:")
for u, v, w in edges:
    a = find(u)
    b = find(v)
    if a != b:
        parent[a] = b
        print(u, "-", v, "Weight:", w)
        total += w
        count += 1
        if count == 4:
            break

if count == 4:
    print("Total weight:", total)
else:
    print("Graph is disconnected")
```

Question 14. Prim's Minimum Spanning Tree using an adjacency matrix

Algorithm

1. Represent campus locations with an adjacency matrix; 0 means no edge between different vertices.
2. Start from vertex 0 and mark it as visited.
3. Find the smallest-weight edge from a visited vertex to an unvisited vertex.
4. Select that edge, mark the new vertex visited and add the weight to the total.
5. Repeat until V - 1 edges are selected or no eligible edge exists.
6. Display the selected edges and total weight. If no eligible edge exists early, the graph is disconnected.

Code

```
g = [
    [0, 2, 4, 0],
    [2, 0, 3, 5],
    [4, 3, 0, 1],
    [0, 5, 1, 0]
]

n = len(g)
used = [False] * n
used[0] = True
total = 0
count = 0

print("Selected edges:")
for _ in range(n - 1):
    minimum = float('inf')
    u = v = -1

    for i in range(n):
        if used[i]:
            for j in range(n):
                if not used[j] and g[i][j] != 0:
                    if g[i][j] < minimum:
                        minimum = g[i][j]
                        u, v = i, j

    if u == -1:
        print("Graph is disconnected")
        break

    print(u, "-", v, "Weight:", minimum)
    used[v] = True
    total += minimum
    count += 1

if count == n - 1:
    print("Total weight:", total)
```

Question 15. Heap Sort in ascending order

Algorithm

1. Start with the input array.
2. Build a max-heap so that each parent is greater than or equal to its children.
3. Swap the root (largest element) with the last element in the heap.
4. Reduce the heap size by one.
5. Restore the max-heap property by heapifying the root.
6. Repeat until the heap size is one.
7. Display the sorted array. Heap Sort takes O(n log n) time and O(1) auxiliary space for this iterative-storage approach, excluding recursion stack; this implementation uses recursive heapify.

Code

```
def heapify(a, n, i):
    largest = i
    left = 2 * i + 1
    right = 2 * i + 2

    if left < n and a[left] > a[largest]:
        largest = left
    if right < n and a[right] > a[largest]:
        largest = right

    if largest != i:
        a[i], a[largest] = a[largest], a[i]
        heapify(a, n, largest)

def heap_sort(a):
    n = len(a)

    for i in range(n // 2 - 1, -1, -1):
        heapify(a, n, i)

    for i in range(n - 1, 0, -1):
        a[0], a[i] = a[i], a[0]
        heapify(a, i, 0)

a = [40, 10, 30, 20, 50]
heap_sort(a)
print("Sorted array:", a)
```

Question 16. Merge Sort for online delivery times

Algorithm

1. Divide the list into two halves.
2. Recursively sort each half until each part has one or zero elements.
3. Compare the first elements of both sorted halves and copy the smaller one into a result list.
4. Continue until one half is exhausted.
5. Append any remaining elements.
6. Return the merged sorted list. The time complexity is O(n log n).

Code

```
def merge_sort(a):
    if len(a) <= 1:
        return a

    mid = len(a) // 2
    left = merge_sort(a[:mid])
    right = merge_sort(a[mid:])

    result = []
    i = j = 0

    while i < len(left) and j < len(right):
        if left[i] <= right[j]:
            result.append(left[i])
            i += 1
        else:
            result.append(right[j])
            j += 1

    result += left[i:]
    result += right[j:]
    return result

orders = [30, 10, 45, 20, 15]
print("Delivery times:", merge_sort(orders))
```

Question 17. Fractional Knapsack

Algorithm

1. Read the weight and profit of each parcel and the truck's capacity.
2. Calculate each parcel's profit-to-weight ratio.
3. Sort parcels in descending order of this ratio.
4. Take each parcel completely if it fits in the remaining capacity.
5. Otherwise, take only the fraction that fits and add its proportional profit.
6. Stop when the capacity is full or all parcels are considered.
7. Display the maximum profit. This greedy strategy solves the fractional, not the 0/1, knapsack problem.

Code

```
weight = [10, 20, 30]
profit = [60, 100, 120]
capacity = 50

items = []
for i in range(len(weight)):
    items.append((profit[i] / weight[i], weight[i]))

items.sort(reverse=True)
total = 0

for ratio, w in items:
    if capacity == 0:
        break
    take = min(w, capacity)
    total += take * ratio
    capacity -= take

print("Maximum profit:", total)
```

Question 18. Naive String Matching

Algorithm

1. Read the text and pattern.
2. Let n be the text length and m be the pattern length.
3. For each starting index from 0 to n - m, compare the text substring of length m with the pattern.
4. If they match, display the starting index.
5. If no index matches, report that the pattern was not found.
6. For an empty pattern (m = 0), every index from 0 through n is a valid match.

Code

```
text = input("Enter text: ")
pattern = input("Enter pattern: ")

n = len(text)
m = len(pattern)
found = False

for i in range(n - m + 1):
    if text[i:i + m] == pattern:
        print("Pattern found at index:", i)
        found = True

if not found:
    print("Pattern not found")
```

Question 19. BFS shortest path in a grid maze

Algorithm

1. Represent free cells as 0 and obstacles as 1.
2. Put the start cell and its path into a queue; mark the start visited.
3. Remove the front entry from the queue.
4. If it is the goal, display its path and stop.
5. Add each valid, free, unvisited neighbouring cell to the queue and record its path.
6. Repeat until the goal is found or the queue becomes empty.
7. If the queue empties first, report that no path exists. BFS finds a shortest path when each move has equal cost.

Code

```
from collections import deque

maze = [
    [0, 0, 1, 0],
    [1, 0, 1, 0],
    [0, 0, 0, 0],
    [0, 1, 1, 0]
]

start = (0, 0)
goal = (3, 3)
q = deque([(start, [start])])
visited = {start}
path = None

while q:
    (r, c), p = q.popleft()

    if (r, c) == goal:
        path = p
        break

    for dr, dc in [(1, 0), (-1, 0),
                   (0, 1), (0, -1)]:
        nr, nc = r + dr, c + dc
        pos = (nr, nc)

        if (0 <= nr < len(maze) and
            0 <= nc < len(maze[0]) and
            maze[nr][nc] == 0 and pos not in visited):
            visited.add(pos)
            q.append((pos, p + [pos]))

if path:
    print("Shortest path:", path)
else:
    print("No path exists")
```

Question 20. A* search in a grid maze using Manhattan distance

Algorithm

1. Represent the maze using 0 for free cells and 1 for obstacles.
2. Set g(start) = 0 and calculate h using Manhattan distance: |row - goal row| + |column - goal column|.
3. Put the start cell in the open list with f = g + h.
4. Remove the entry with the smallest f value.
5. If it is the goal, display the path and stop.
6. Otherwise, examine valid neighbouring cells and add improved paths with their g, h and f scores.
7. Continue until the goal is reached or the open list becomes empty. If empty, report no path.

Code

```
maze = [
    [0, 0, 1, 0],
    [1, 0, 1, 0],
    [0, 0, 0, 0],
    [0, 1, 1, 0]
]

start = (0, 0)
goal = (3, 3)

def h(pos):
    return abs(pos[0] - goal[0]) + abs(pos[1] - goal[1])

open_list = [(h(start), 0, start, [start])]
best_cost = {start: 0}
path = None

while open_list:
    open_list.sort(key=lambda x: x[0])
    f, cost, pos, p = open_list.pop(0)

    if cost != best_cost.get(pos):
        continue

    if pos == goal:
        path = p
        break

    r, c = pos
    for dr, dc in [(1, 0), (-1, 0),
                   (0, 1), (0, -1)]:
        nr, nc = r + dr, c + dc
        nxt = (nr, nc)

        if (0 <= nr < len(maze) and
            0 <= nc < len(maze[0]) and
            maze[nr][nc] == 0):
            new_cost = cost + 1

            if new_cost < best_cost.get(nxt, float('inf')):
                best_cost[nxt] = new_cost
                open_list.append(
                    (new_cost + h(nxt), new_cost,
                     nxt, p + [nxt])
                )

if path:
    print("Path:", path)
else:
    print("No path exists")
```